# The Java Threading API

16 November 2025 10:13

**Threading Using the Thread Class**

We considered threads as separate task that execute in parallel. These tasks are simply code executed by the thread, and this code is actually part of our program. The code may download an image from the server or may play audio file on the speakers or any other task; because it is a code; it can be executed by our original thread. To introduce the parallelism we desire, we must create a new thread and arrange for the new thread to execute the appropriate code.

```
public clas OurClass{
     public void run(){
        for(int I = 0; I < 100; I++){
           System.out.println("HELLO");
        }
     }
}
```

'java.lang.Thread'

```
public clas OurClass extends Thread{
     public void run(){
        for(int I = 0; I < 100; I++){
           System.out.println("HELLO");
        }
     }
}
```

To start the thread we need start() method.

```
package MyThreads;
public class TestFirstThread {
    public static void main(String[] args) {
        FirstThread t = new FirstThread();
        t.start();
        for(int i = 0; i < 10; i++){
            System.out.println("Main");
        }
    }
}
```

Main. → executing

t → borned; and ready.

start() → it will call the run() method is called.

calls the run() method defined in this thread class.

```
package MyThreads;
public class FirstThread extends Thread {
    @Override
    public void run(){
        int max = 20;
        for (int i = 0; i < max; i++) {
            System.out.println(i);
        }
    }
}
```

The method that the newly created thread will execute. Developers should override this method with the code they want new thread to run.

run() is essentially empty method in Thread class.

**run() Versus main()**

In essence, the run() Method may be thought of as the main() method of the newly formed thread: A new thread begins execution with a run() method in the same way a program begins execution with the main() method.

While the main () method receives its argument from argv parameter (which is typically set from the command line) The newly created thread must receive its arguments programmatically from the originating thread. Hence parameters can be passed in via the constructor, static instance variable or any other technique designed by the developer.

**sleep() methods:**

**static void sleep(long milliseconds) throws InterruptedException:** Puts the currently executing thread to sleep for the specified number of milliseconds. This method is static and may be accessed through the Thread class name.

**static void sleep(long milliseconds, int nanoseconds) throws InterruptedException:** Puts the currently executing thread to sleep for the specified number of milliseconds and nanoseconds. This method is static and may be accessed through the thread class name.

```java
package MyThreads;
public class TestFirstThread {
    public static void main(String[] args) throws InterruptedException {
        FirstThread t = new FirstThread();
        t.start();
        int i;
        for( i = 0; i < 10; i++){
            System.out.println("Main"+ i);
            Thread.sleep(1000);
        }
    }
}
```

```java
package MyThreads;
public class FirstThread extends Thread {
    @Override
    public void run(){
        final int max = 10;
        for (int i = 0; i < max; i++) {
            System.out.println(i);
            try {
                Thread.sleep(1500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    }
}
```

Output:

```
0
Main0
Main1
1
Main2
2
Main3
Main4
3
Main5
4
Main6
Main7
5
Main8
6
Main9
7
8
9
```

**Thread Using the Runnable Interface**

As simple as it to create another thread of control. There is one problem with the technique we have outlined so far. It's caused by the fact that Java classes can inherit their behavior only from a single class, which means that inheritance itself can be considered a scarce resource and is therefore "expensive" to the developer.

In our example we are threading a simple loop, so this is not much of a concern, however. If we have a complete class structure that already has a detailed inheritance tree and want it to run in its own thread, we cannot simply make this class structure inherit from the Thread class as we did before. One solution would be to create a new class that inherits from the thread and contains references to the instances of the class we need. This level of indirection is an annoyance.

The Java language deals with this lack of multiple inheritance by using the mechanism known as interfaces. This mechanism is supported by the Threat class and simply means that instead of inheriting from the Thread class, we can implement the Runnable interface (java.lang.Runnable) Which is defined as follows:

```
public interface Runnable{
   public abstract void run();
}
```

The Runnable interface contains only one method, the run () method. The Thread class actually implements the Runnable interface. Hence, when you inherit from Thread class, your subclass also implements the Runnable interface. However, in this case we want to implement the Runnable interface without actually inheriting from the Thread class. This is achieved by simply substituting the phrase **"implements Runnable"** for the phrase **"extend thread"**, no other changes are necessary in our thread creation process.

```java
package MyThreads;
public class FirstThread implements Runnable {
    @Override
    public void run(){
        final int max = 10;
        for (int i = 0; i < max; i++) {
            System.out.println(i);
            try {
                Thread.sleep(1500);
            } catch (InterruptedException e) {
                e.printStackTrace();
```

```
            }
        }
    }
}
```

```
package MyThreads;
public class TestFirstThread {
    public static void main(String[] args) throws InterruptedException {
        FirstThread ft = new FirstThread();
        Thread t1 = new Thread(ft);
        t1.start();
        int i;
        for( i = 0; i < 10; i++){
            System.out.println("Main"+ i);
            Thread.sleep(1000);
        }
    }
}
```

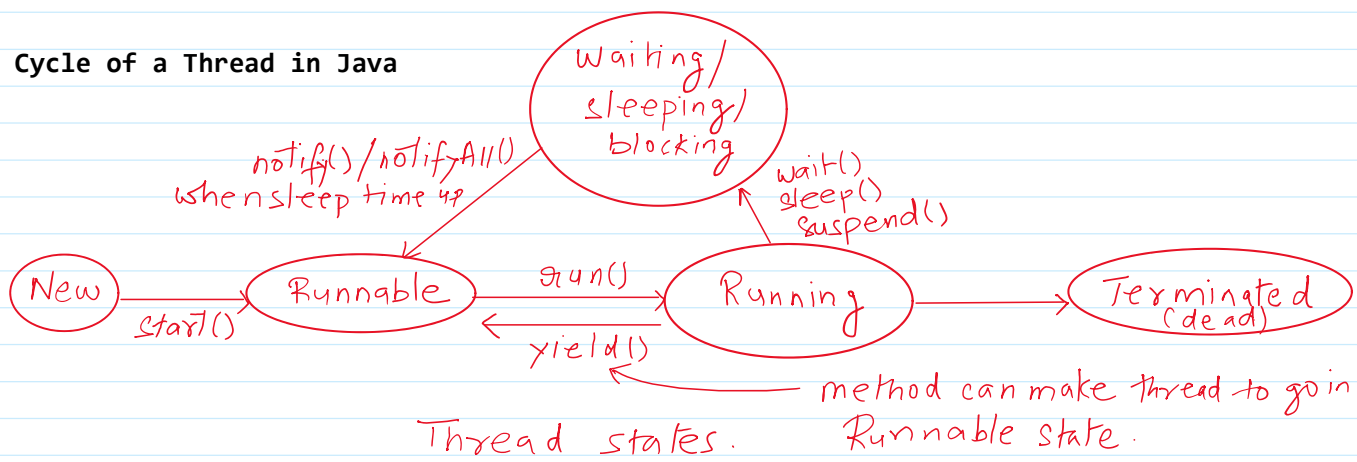
← Object of user defined Thread class.

← Object of Thread class is created, and we have passed the object of user-defined Thread class. To get the access of start() method of Thread class, which calls the run() method of user-defined class.

Why ft.run(); method directly?

If the, run() method, from the user-defined Thread class is called directly, it will not initiate the thread, it will be a normal method call.

**Life Cycle of a Thread in Java**




A thread in Java goes through several states from creation to completion.
These states are defined in the Thread.State enum.

```
public static enum Thread.State
extends Enum<Thread.State>
```

A thread state. A thread can be in one of the following states:

- NEW
  A thread that has not yet started is in this state.
- RUNNABLE
  A thread executing in the Java virtual machine is in this state.
- BLOCKED
  A thread that is blocked waiting for a monitor lock is in this state.
- WAITING
  A thread that is waiting indefinitely for another thread to perform a particular action is in this state.
- TIMED_WAITING
  A thread that is waiting for another thread to perform an action for up to a specified waiting time is in this state.
- TERMINATED
  A thread that has exited is in this state.

A thread can be in only one state at a given point in time. These states are virtual machine states which do not reflect any operating system thread states.

**Since:**

1.5

**See Also:**

Thread.getState()

1. NEW state: A thread is in the new state when it is created but not started
   Example:
   ```
   Thread t = new Thread(); //New State
   ```
   At this point:
   - Memory is allocated.
   - Thread object exists
   - But the thread is not scheduled to run.

2. RUNNABLE State: When we call start () On a thread, it enters the runnable state.
   ```
   t.start(); //runnable state
   ```
   This means:
   - The thread is ready to run.
   - It is placed in the thread schedular queue.
   - It may or may not execute immediately.

3. RUNNING state: A thread enters the running state when the thread scheduler picks it to run.
   Inside the run() method:

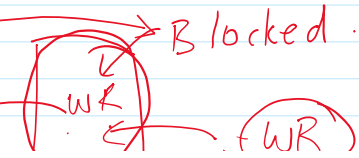
   ```
   public void run(){
      System.out.print("\n Thread is running…");
   }
   ```
   The CPU decides when a thread actually runs-not your code.

4. WAITING/BLOCKED/TIMED_WAITING States: These states occur when a thread is temporarily inactive but not finished.
   a. WATING state: Thread waits indefinitely for another thread to perform an action.
      Example:
      ```
      Obj.wait();//waiting thread can be awaked by(runnable) notify() or notifyAll()
      ```
      This method belongs to Object class. When your thread object is synchronized then use wait() method.

      ```
      t.join(); //Waiting
      ```
      This method belongs to Thread class.
      Ensures sequential execution by causing the current thread to pause its execution until the thread on which join() is called completes its execution.
      Can be called with or without a synchronized context. It does not involve acquiring or releasing object locks.

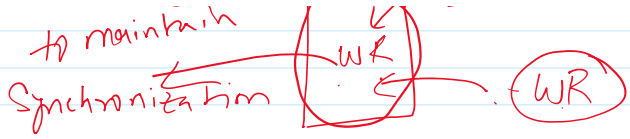
   b. TIMED_WAITING State: Thread waits for specific time
      Examples:
      ```
      Thread.sleep(1000); //Timed waiting
      ```

why to maintain synchronization



b. TIMED_WAITING State: Thread waits for specific time
   Examples:
      Thread.sleep(1000); //Timed waiting
      Obj.wait(5000); //Timed waiting
      t.join(2000);   //Timed waiting

to maintain Synchronization




c. BLOCKED State: Thread is blocked because it wants a monitor lock that another thread holds. Occurs in synchronized code.
   Example:
      synchronized(obj){
        //Code that acquires lock
      }
   If another thread is using obj, current thread becomes, BLOCKED.

5. TERMINATED State: A thread enters the terminated (dead) state when:
   i. run() method finishes normally
   ii. Thread is stopped using stop()  (unsafe, deprecated)
   iii. An uncaught exception occurs inside run()

```java
package MyThreads;
public class MyNewThread extends Thread{
    public void run(){
        System.out.println("Thread is running...");

        try {
            System.out.println("Thread goes to Timed Waiting(sleep)...");
            Thread.sleep(2000);
            System.out.println("Thread is running again...");
        } catch (InterruptedException e) {
            System.err.println("Exception: " + e);
        } //Timed waiting
        System.out.println("Thread is terminated");
    }
}
```

```java
package MyThreads;
public class ThreadLifeCycle {
    public static void main(String[] args) {
        MyNewThread t = new MyNewThread();
        System.out.println("Thread state after creation: " +t.getState());
        t.start();
        System.out.println("Thread state after t.start(): " +t.getState());
        try {
            Thread.sleep(1000);
            System.out.println("Thread state while running: " + t.getState());
            t.join(); //Main thread waits
            System.out.println("Thread state after join(): " + t.getState()); //TERMINATED
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

```
Thread state after creation: NEW
Thread state after t.start(): RUNNABLE
Thread is running...
Thread goes to Timed Waiting(sleep)...
Thread state while running: TIMED_WAITING
Thread is running again...
Thread is terminated
Thread state after join(): TERMINATED
```